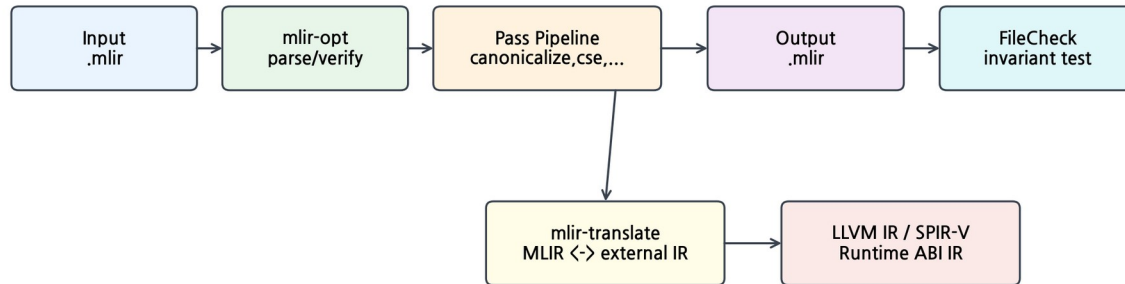


MLIR NPU Compiler - Lecture 04

mlir-opt, mlir-translate, FileCheck: pass pipeline execution and regression testing

NPU Compiler Tooling Loop: transform -> inspect -> test -> regress



4강 목표: command line tool을 익혀 pass pipeline을 반복 실행하고, FileCheck로 깨지지 않는 회귀 테스트를 작성한다.

1. 강의 목표

- mlir-opt 를 사용해 MLIR input 을 parse/verify 하고 pass pipeline 을 실행한다.
- textual pass pipeline 문법을 읽고 NPU lowering pipeline 을 command line 으로 표현한다.
- mlir-translate 의 역할을 mlir-opt 와 구분한다: MLIR 내부 변환 vs 외부 IR translation.
- FileCheck 와 lit RUN line 을 사용해 pass regression test 를 작성한다.
- NPU compiler 에서 fusion, tiling, bufferization, command emission 의 invariants 를 테스트 항목으로 바꾼다.

2. 20 강 커리큘럼에서 4 강의 위치

이전 강의	이번 강의	다음 강의로 연결
1-3 강: MLIR big picture, Operation/Value/Type/Attribute, Region/Block 구조	4 강: MLIR toolchain 을 이용해 IR 을 실행, 관찰, 회귀 테스트하는 방법	5 강 이후: Dialect/Pass/Rewrite/Conversion 을 직접 만들 때 테스트 harness 로 계속 사용

3. mlir-opt 의 역할

mlir-opt 는 MLIR 코드를 입력으로 받아 parser 와 verifier 를 거친 뒤, 선택적으로 pass sequence 를 실행하고 다시 IR 로 출력하는 command-line tool 입니다. NPU compiler 개발에서는 custom pass 를 빠르게 반복 실행하고, pass 전후 IR 을 관찰하며, regression test 를 만드는 핵심 도구입니다.

```
# 1) parse + verify 만 수행
mlir-opt input.mlir

# 2) pass pipeline 실행
mlir-opt input.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))'

# 3) IR 변화 디버깅
mlir-opt input.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --mlir-print-ir-after-all
```

4. Pass pipeline 문법

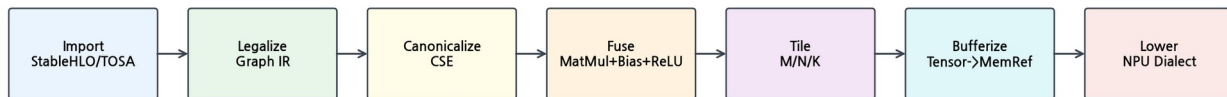
textual pass pipeline 은 op-anchor 와 pass list 의 중첩 구조로 읽습니다. NPU compiler 에서는 graph/module level pass 와 function/kernel level pass 가 다르므로 anchor 를 잘못 잡으면 pass 가 실행되지 않거나 너무 넓은 범위에 적용됩니다.

```
pipeline ::= op-anchor '(' pipeline-element (',' pipeline-element)* ')'
```

```
builtin.module(
  func.func(cse,canonicalize),
  convert-func-to-llvm{use-bare-ptr-memref-call-conv=1}
)
```

구성 요소	의미	NPU compiler 예시
builtin.module	module operation 에 anchored 된 pass manager	frontend graph legalize, symbol/global analysis
func.func	function/kernel body 단위 nested pass	kernel fusion, loop transform, vectorization
any	viable operation 이면 실행되는 op-agnostic anchor	prototype debugging 에서만 신중히 사용
pass options	pass 별 key=value 설정	npu-tile{tile-m=16 tile-n=16 tile-k=32}

NPU Lowering Pipeline을 mlir-opt pass-pipeline 문자열로 표현



Example: builtin.module(func.func(canonicalize,cse), convert-..., npu-fuse, npu-tile{tile-m=16 tile-n=16 tile-k=32})

5. NPU lowering pipeline 을 command line 으로 표현하기

실제 NPU compiler 는 보통 하나의 pass 가 모든 일을 하지 않습니다. Legalize, canonicalize, fusion, tiling, bufferization, memory planning, codegen pass 가 순서대로 연결됩니다. 4 강에서는 이 순서를 명령어 문자열로 명시하고, 어느 stage 가 어떤 IR invariant 를 보장하는지 기록합니다.

```
# Pseudo command for a future custom NPU tool.
npu-opt model.mlir --pass-pipeline='builtin.module(
  npu-legalize-frontend,
  func.func(canonicalize,cse,npu-fuse,npu-tile{tile-m=16 tile-n=16 tile-k=32}),
  npu-bufferize,
  npu-lower-to-command-buffer
),'
```

6. 디버깅에 유용한 mlir-opt flags

Flag	용도	NPU compiler 활용
--mlir-print-ir-after-all	각 pass 이후 IR dump	어느 pass 에서 layout/dma/barrier 가 깨졌는지 추적
--mlir-print-ir-after-change	IR 이 바뀐 pass 이후만 dump	대형 Transformer graph 디버깅 로그 축소
--mlir-print-ir-tree-dir=<dir>	pass 별 IR dump 를 directory tree 에 저장	CI artifact 로 pass stage output 보관
--mlir-timing	pass 별 실행 시간 측정	compiler latency 병목 분석
-split-input-file	한 파일에 여러 test case 분리	positive/negative case 를 같은 파일에 정리
-verify-diagnostics	expected-error/remark 검증	unsupported op/layout/tile size diagnostic test
--mlir-pass-pipeline-crash-reproducer=<file>	crash reproducer 생성	custom NPU pass crash 최소 재현

7. mlir-translate 의 역할

mlir-opt 가 MLIR 내부의 pass pipeline 실행 도구라면, mlir-translate 는 MLIR module 과 외부 표현 사이의 translation 을 담당합니다. 대표적으로 LLVM dialect 로 완전히 lowering 된 MLIR 을 LLVM IR 로 내보내거나, SPIR-V binary 같은 외부 형식과 왕복할 때 사용합니다.

```
# LLVM dialect MLIR -> LLVM IR
mlir-translate --mlir-to-llvmir llvm_dialect.mlir > output.ll

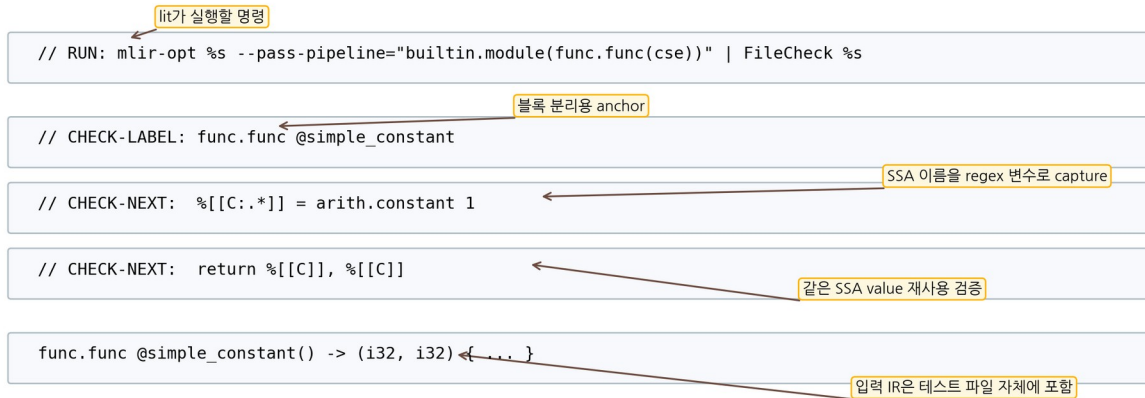
# 일반적인 흐름
high_level.mlir --mlir-opt passes--> llvm_dialect.mlir --mlir-translate--> output.ll
```

NPU compiler 에서는 최종 backend 가 LLVM 이 아닐 수도 있습니다. 그러나 custom command buffer, ELF note, firmware packet, simulator JSON 같은 외부 representation 으로 나가는 경계는 mlir-translate 와 유사한 책임으로 설계할 수 있습니다.

8. FileCheck 기본

FileCheck 는 tool output 을 exact golden file 처럼 비교하지 않고, 필요한 pattern 만 지정해 검증합니다. MLIR pass output 은 SSA 이름, attribute ordering, whitespace 가 바뀔 수 있으므로 회귀 테스트는 semantic invariant 를 중심으로 작성해야 합니다.

FileCheck Test Anatomy



Directive	의미	실무 팁
CHECK-LABEL	독립 테스트 블록 시작점	함수명 또는 kernel 명에 사용
CHECK	순서가 있는 positive match	DMA -> barrier -> compute 처럼 순서가 의미 있을 때 사용
CHECK-NEXT	바로 다음 줄	너무 많이 쓰면 brittle 해짐
CHECK-SAME	같은 줄 match	attribute/type 일부 확인에 유용
CHECK-NOT	두 positive match 사이 부재	lowering 후 illegal op 제거 확인
CHECK-DAG	순서 무관 match	canonicalization 후 독립 constant/metadata 확인
%[[:.*]]	FileCheck 변수 capture	SSA numbering 변화에 강한 테스트 작성

9. 좋은 regression test 작성 원칙

- diff test 가 아니라 invariant test 를 작성한다. 출력 전체를 고정하지 말고 pass 가 보장해야 하는 핵심만 확인한다.
- 함수명은 test intent 를 드러내야 한다. 예: `matmul_bias_relu_f32`, `negative_bad_tile_k_not_multiple`.
- positive case 와 negative case 를 모두 둔다. NPU pass 는 지원하지 않는 shape/layout 에 대해 조용히 잘못 낮추면 안 된다.

- CHECK-DAG 는 order 가 의미 없는 op 에만 사용한다. DMA/barrier/compute 순서는 일반 CHECK 순서로 강제한다.
- quantization, memory space, layout, async token, tensor descriptor field 는 NPU compiler correctness 에 직접 연결되므로 테스트 대상이다.

10. NPU compiler regression matrix

NPU Compiler Regression Test Matrix

Pass Stage	Positive Test	Negative Test	NPU Invariant
Legalize	supported op lowers	unsupported op emits error	no illegal graph op remains
Fuse	matmul+bias+relu fused	shape/layout mismatch not fused	activation order preserved
Tile	M/N/K tile loops appear	bad tile size rejected	tile covers full tensor
Bufferize	tensor -> memref	alias conflict diagnosed	memory space preserved
Codegen	npu.dma/npu.matmul emitted	missing barrier rejected	DMA before compute

NPU compiler 에서는 numerical correctness 뿐 아니라 command ordering 과 memory correctness 가 중요합니다. 따라서 FileCheck 테스트는 단순히 op 가 나왔는지뿐 아니라, op 의 순서, memory space, tile attribute, quantization metadata, barrier/await 관계를 함께 점검해야 합니다.

11. 실습 A - mlir-opt 로 parse/verify 확인

```
cd examples
mlir-opt 01_parse_verify.mlir
mlir-opt 01_parse_verify.mlir | FileCheck 01_parse_verify.mlir
```

목표: pass 없이도 parser/verifier 가 실행된다는 것을 확인합니다. input IR 이 잘못된 경우 pass 단계 이전에 오류가 발생해야 합니다.

12. 실습 B - CSE pass 와 FileCheck 변수

```
mlir-opt 02_cse_regression_test.mlir --pass-pipeline='builtin.module(func.func(cse))' | FileCheck 02_cse_regression_test.mlir
```

목표: 두 개의 동일 constant 가 하나로 합쳐지고, return 이 같은 SSA value 를 두 번 사용하는지 확인합니다. '%[[C:.*]]'는 printed SSA 이름에 의존하지 않기 위한 FileCheck 변수입니다.

13. 실습 C - pass pipeline debug

```
mlir-opt 03_pass_pipeline_debug.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --mlir-print-ir-after-all
```

```
mlir-opt 03_pass_pipeline_debug.mlir --pass-pipeline='builtin.module(func.func(cse,canonicalize))' --mlir-print-ir-after-change
```

목표: pass sequence 중 어느 pass 가 실제로 IR 을 변경했는지 관찰합니다. 대형 AI graph 에서는 `after-change`가 로그 폭 발을 줄이는 데 유용합니다.

14. 실습 D - negative diagnostic test

```
mlir-opt 05_verify_diagnostics_negative.mlir -split-input-file -verify-diagnostics
```

목표: malformed IR 또는 unsupported input 이 반드시 오류를 발생시키는지 검증합니다. NPU compiler 에서 invalid tile size, unsupported quantization, illegal layout 을 조용히 accept 하면 실리콘/런타임에서 디버깅이 매우 어려워집니다.

15. 실습 E - NPU pass test template 작성

```
// RUN: npu-opt %s --pass-pipeline='builtin.module(func.func(npu-fuse,npu-tile{tile-m=16 tile-n=16 tile-k=32}))' | FileCheck %s
// CHECK-LABEL: func.func @matmul_bias_relu
// CHECK: npu.matmul
// CHECK-SAME: tile_m = 16
// CHECK-SAME: tile_n = 16
// CHECK: npu.relu
// CHECK-NOT: stablehlo.dot_general
```

목표: 아직 custom NPU dialect 가 없어도 어떤 invariant 를 확인해야 하는지 먼저 테스트로 설계합니다. 이후 18 강에서 실제 ODS 기반 npu dialect 를 만들 때 이 템플릿을 발전시킵니다.

16. 흔한 실수와 교정

실수	문제	교정
출력 전체를 CHECK-NEXT 로 고정	printer 변경만으로 test failure	핵심 invariant 만 regex/변수로 확인
SSA 이름을 직접 확인	canonicalizer/renaming 에 취약	%[[V:.*]] capture 사용
CHECK-DAG 로 순서 있는 command 를 확인	DMA/barrier bug 를 숨김	order 가 correctness 이면 CHECK 순서 사용
negative case 없음	unsupported case 가 silent miscompile 가능	-verify-diagnostics test 추가
pipeline anchor 생략/오류	pass 가 기대 위치에서 실행되지 않음	builtin.module/func.func/any 를 명시적으로 설계

17. 과제

다음 pseudo pass 를 가정하고 FileCheck 기반 regression test 를 작성하세요: `npu-fuse-matmul-bias-relu`, `npu-tile{tile-m=16 tile-n=16 tile-k=32}`, `npu-lower-to-command-buffer`.

- positive test: matmul + bias + relu 가 fused 또는 순서 보존 lower 되는지 확인
- negative test: bias shape mismatch 가 diagnostic 으로 잡히는지 확인
- command order test: DMA load -> barrier/await -> matmul -> DMA store 순서 확인
- 설명: 각 CHECK line 이 어떤 NPU correctness invariant 를 검증하는지 주석으로 작성

18. References

- MLIR Using mlir-opt: <https://mlir.llvm.org/docs/Tutorials/MlirOpt/>
- MLIR Pass Infrastructure: <https://mlir.llvm.org/docs/PassManagement/>
- MLIR Passes: <https://mlir.llvm.org/docs/Passes/>
- MLIR Testing Guide: https://mlir.llvm.org/getting_started/TestingGuide/
- LLVM FileCheck Command Guide: <https://llvm.org/docs/CommandGuide/FileCheck.html>
- mlir-translate source/main entry: <https://github.com/llvm/llvm-project/blob/main/mlir/tools/mlir-translate/mlir-translate.cpp>